SQL PROJECT

Supply Chain

Database Analysis

Exploring customer behaviour, supplier performance & order trends using structured SQL queries

830 Orders	91 Customers	78 Products	29 Suppliers

Project Overview

🎯 What is this project?

A complete MySQL project that analyses a multi-table Supply Chain database.

It covers 4 levels of SQL difficulty:

Level 1 Basic Queries

Level 2 Intermediate + Window Functions

Level 3 CTEs & Complex Joins

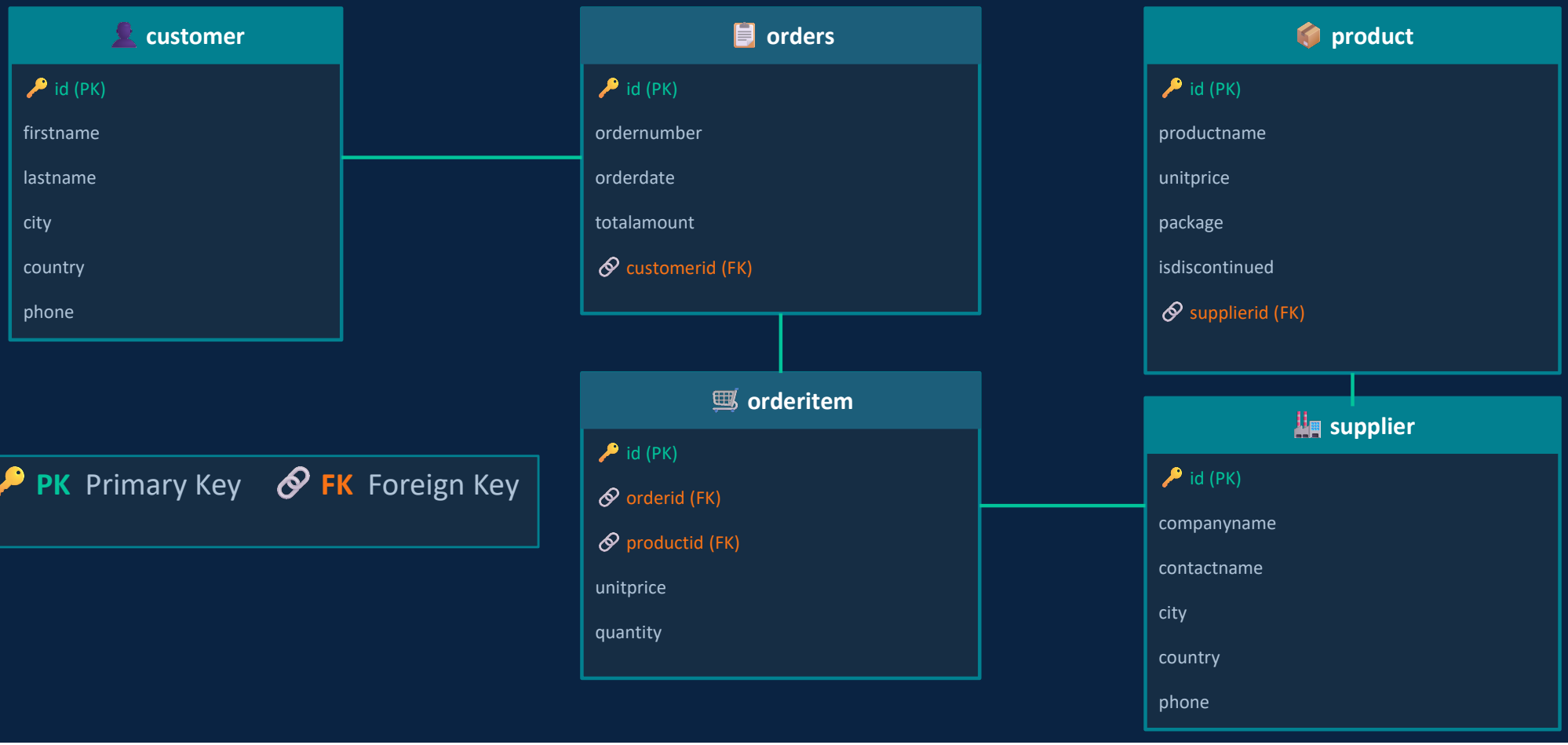
Level 4 Business Logic & Calculations

📦 The Dataset

customer	91 customers from 21 countries
supplier	29 suppliers across 16 countries
product	78 products, various categories
orders	830 orders (Jul 2012 – May 2014)
orderitem	2155 items with qty & price

Database Schema

How the 5 tables connect



PK Primary Key FK Foreign Key

Level 1 — Basic Queries

SELECT · WHERE · GROUP BY · ORDER BY

Customer Demographics

Count customers per country — USA, Germany & Brazil top the list

High-Value Items

Subquery finds the most expensive product each customer ever ordered

Active Inventory

Filter products where isdiscontinued = 0 to see live stock

Top Supplier

Supplier ID 12 supplies the most products (7 items)

Supplier Catalog

JOIN product + supplier to see who supplies what

Order Trends

DATE_FORMAT groups orders by month — peak in late 2013

Regional Search

WHERE country = 'Mexico' returns all Mexican customers

```
SELECT COUNT(id), country FROM customer GROUP BY country ORDER BY 1 DESC;
```

Level 2 — Intermediate Queries

JOINs · Aggregations · Window Functions · LIMIT

📊 Product Demand Rank

RANK() window function sorts products by total quantity ordered. Top products drive 40%+ of order volume.

💰 Revenue Analysis

Total revenue calculated per year using SUM(totalamount). Revenue grew consistently from 2012 → 2014.

👑 Top Spender

Customer #63 (Francisco Chang) is the biggest spender across all orders — identified via SUM + ORDER BY.

🏆 Supplier Revenue Top 3

Top 3 suppliers identified by joining supplier → product → orderitem and summing unit_price × quantity.

📅 Order History

LAG() window function compares each order's date to the customer's previous order — shows purchase frequency.

🔗 Order Details

3-table JOIN (orders + product + supplier) displays every order with full product and supplier context.

RANK() OVER (ORDER BY SUM(quantity) DESC) | **LAG(orderdate) OVER** (PARTITION BY customerid ORDER BY orderdate)

Level 3 — Advanced Queries

WITH (CTE) · Complex JOINs · Subqueries

01

Bulk Buyers

```
HAVING COUNT(DISTINCT productid) > 10
```

Finds customers who ordered more than 10 different products in a single order — high-engagement buyers worth targeting.

02

Small Orders

```
WHERE oi.quantity = 1
```

Lists all product details where exactly 1 unit was ordered — useful for identifying trial purchases or slow-moving products.

03

Premium Suppliers

```
WHERE p.unitprice > 100
```

JOIN supplier + product and filter for unitprice > 100. Côte de Blaye (263.50) and Thüringer Rostbratwurst (123.79) stand out.

04

City-Based Networking

```
GROUP BY city, country HAVING COUNT(*) > 1
```

Finds customers who share the same city — e.g. multiple customers in London, México D.F., Sao Paulo for B2B analysis.

Level 4 — Expert Queries

Complex Calculations · Business Logic · Market Research

💸 Savings Analysis

The company offers discounts where the selling price in orderitem differs from the listed unitprice in product.

Formula: (product.unitprice - orderitem.unitprice) × quantity

Orders are ranked by total savings using RANK() OVER (ORDER BY savings DESC).

🔬 Market Research (Mr. Kavin)

A. High-Demand Products
Identify products with the highest total quantity ordered — these are the best candidates for a new supplier to stock.

B. Competitor Analysis
For each high-demand product, find which existing suppliers already provide it — showing Mr. Kavin who he would compete against.

SQL Pattern — CTE + Window Function

```
WITH savings_cte AS ( SELECT o.id, SUM((p.unitprice - oi.unitprice) * oi.quantity) AS savings
  FROM orders o  JOIN orderitem oi ON o.id=oi.orderid  JOIN product p ON oi.productid=p.id )
SELECT *, RANK() OVER (ORDER BY savings DESC) AS savings_rank  FROM savings_cte;
```

Key Business Insights

What the data tells us

#1 & #2

USA & Germany

lead customer count with the most buyers

7 Products

Supplier 12

Plutzer Lebensmittelgroßmärkte supplies the most diverse product range

$263.50

Côte de Blaye

Most expensive single product — a premium French wine

Top Spender

Customer #63

Francisco Chang in México D.F. has the highest total order value

Peak Month

Dec 2013

Highest number of orders placed — strong year-end demand

1 View

5 Tables

jointables VIEW simplifies all complex queries into a single reusable join

GitHub Repository Structure

How the project is organized

```
📁 supply-chain-sql/
├── 📄 README.md
├── 📁 sql/
│   ├── 01_schema.sql
│   ├── 02_data_insert.sql
│   ├── 03_level1_basic.sql
│   ├── 04_level2_intermediate.sql
│   ├── 05_level3_advanced.sql
│   └── 06_level4_expert.sql
├── 📁 diagrams/
│   └── ER_diagram.mwb
└── 📁 presentation/
    └── supply_chain.pptx
```

README.md

Project description, setup guide, query explanations and key findings

01_schema.sql

CREATE TABLE statements for all 5 tables with FK constraints

02_data_insert.sql

All INSERT statements: customers, suppliers, products, orders & items

03-06 .sql

Grouped by difficulty level for easy navigation and learning

ER_diagram.mwb

MySQL Workbench diagram showing all table relationships visually

Tools & Technologies

MySQL

Core database engine for all CREATE, INSERT and SELECT operations

MySQL Workbench

GUI tool used for schema design, ER diagram and query execution

GitHub

Version control and hosting for all SQL files and documentation

PowerPoint

Presentation of findings, schema diagrams and query explanations

SQL Concepts

SELECT, WHERE, JOIN, GROUP BY, HAVING, Window Functions, CTEs

Database Design

Normalization, primary keys, foreign keys and relational schema

Thank You!

Questions & Discussion

📁 **GitHub Repo:** github.com/your-username/supply-chain-sql

🛠️ **Tech Stack:** MySQL · MySQL Workbench · GitHub

Level 1: Basic | Level 2: Intermediate | Level 3: Advanced | Level 4: Expert